

which data to cache and for how long. Networking libraries usually have inbuilt support for such headers and cache responses automatically. The backend can also be designed to reduce the amount of data transfer by comparing the version numbers (or Etags) of resources and sending a representation only if the resource has changed.

## Offline updates

Data once cached can be displayed in offline mode easily, but what about adding/editing data? You may want to allow users to register, post pictures and update profiles, even when offline.

## Queuing

Whenever the application does not have a network connection, the newly entered or edited data operations can be collected in a local queue and processed later. A user should be able to use the app normally and always see the updated data, even if it is unprocessed. The queue also helps maintain the correct sequence of actions, which could be crucial at times. Imagine how confusing a conversation would be if the messages arrived in a random order.

A key point is to always inform users about queued operations. It would be better to create a separate UI that displays a list of unprocessed changes, giving users an option to cancel or retry them. As the operation is performed, notify users about the respective success or failure outcomes. This will help build more trust in the app among users—an assurance that it won't lose any data regardless of the connectivity state.

## Synchronisation

When the signal is restored, the collected changes must be synchronised with data in the backend. Synchronisation can be triggered in various ways—as soon as the network is available, on launching the application, once in a day, after every six hours, etc. It's better to go with bundled data transfers to help conserve battery resources.

## Sync conflicts

While the user was working offline, it is possible that changes were made by other users to the same data. Or, the same user can change his data from multiple sources/devices, which can lead to conflicts that are usually detected at the level of a row in a database. A row is in conflict if it is changed at more than one of the sources between synchronisations. It could be a unique key collision, in which a row with the same unique key is inserted from different sources, or an update collision when the same row is updated from different places. It could also be a delete conflict when updating a row that has been deleted already.

Let's look at an example.

1. Clients A and B synchronise with the server and pull version v1 of a resource. The resource can be represented as a row in the database.

2. A updates the resource and synchronises with the server. There is no conflict. The version is updated to v2.
3. Later, B updates the same resource (at v1) and synchronises. A write conflict occurs because it is an update on the older version and the resource has been updated to v2 already.

Sometimes, conflicts can be detected on the client side. In the example above, if before pushing the changes, B pulls the resource again, it can compare the versions and see there is a conflict. B can then choose to discard its local changes or apply some other resolution, and inform the user accordingly.

Just like the version control systems, these conflicts must be resolved before the synchronisation process gets completed. There can be different conflict resolution strategies and choosing the right one is important to prevent data inconsistency problems.

**Server wins:** In this approach, changes received during synchronisation are discarded, leaving data in the database unchanged.

**Client wins:** The conflict is ignored and changes received are accepted, overwriting the value in the database. One should be very careful when using this approach as it might lead to data inaccuracy. Incorrectly overwriting a change of location (from Delhi to Mumbai, for instance) will result in showing completely irrelevant recommendations to the user.

**Last update wins:** Data with the most recent modified timestamp wins.

**Let the user decide:** Leave the decision to app users. The users can view the conflicting values and decide which one to keep.

**Custom algorithm:** A custom algorithm defined on the basis of business rules can be used to resolve conflicts. For example, there could be a business rule that says, "Pick the largest of the two values or take the union of the conflicting data sets and merge them."

You may encounter situations where data is complex. Depending on your implementation, you can choose different approaches and build a robust conflict resolution system.

Synchronising offline data makes the application more responsive and optimised. You can build your own system for managing data transfers or use frameworks like Android's Sync Adapter that automate it for you.

Lack of connectivity is not an error condition. Instead of showing an error message, the app should be tailored to provide a seamless experience, always. After all, we all know a happy customer is the best customer! 

By: Minni Arora

The author works as a technical architect with Naukri.com. She has nine years of experience in architecture, design and development of web as well as mobile applications. She can be contacted at [minienarula@gmail.com](mailto:minienarula@gmail.com).